

Sistema de Gestión Veterinaria

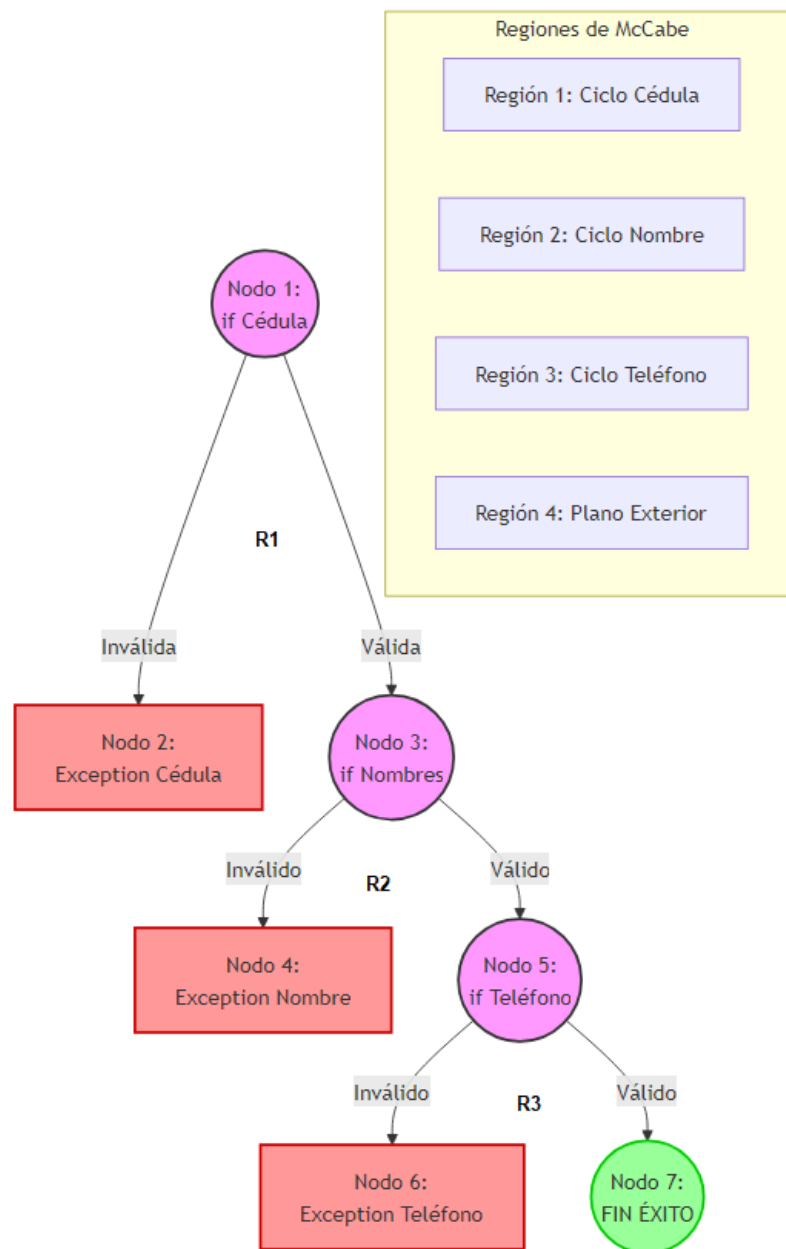
Módulo 1 – Clientes / Mascotas

Integrante: Galo Izquierdo V.

MÉTODO 1: validarCliente(ModeloCliente c)

```
1: private void validarCliente(ModeloCliente c) throws Exception {
2:     if (c.getCedula() == null || !c.getCedula().matches("\\d{10}")) {
3:         throw new Exception("Cédula inválida");
4:     }
5:     if (c.getNombres() == null || !c.getNombres().matches("[a-zA-Z ]+")) {
6:         throw new Exception("Nombre inválido");
7:     }
8:     if (c.getTelefono() == null || !c.getTelefono().matches("\\d+")) {
9:         throw new Exception("Teléfono inválido");
10:    }
11: }
```

A. Grafo de Flujo de Control



[1] Inicio del método

|

v

[2] Decisión: ¿c.getCedula() == null o inválida?

/ \

(Sí) / \ (No)

v v

[3] Throw Exception [4] Decisión: ¿c.getNombres() == null o inválido?

/ \
(Sí) / \ (No)
v v

[5] Throw Exception [6] Decisión: ¿c.getTelefono() == null o inválido?

/ \
(Sí) / \ (No)
v v

[7] Throw Exception [8] Fin del método (Validación Exitosa)

Enumeración de Nodos Clínicos

- Nodo 1: Inicio del método y evaluación del primer if (Cédula).
- Nodo 2: Lanzamiento de excepción por Cédula inválida (throw new Exception).
- Nodo 3: Evaluación del segundo if (Nombres).
- Nodo 4: Lanzamiento de excepción por Nombre inválido (throw new Exception).
- Nodo 5: Evaluación del tercer if (Teléfono).
- Nodo 6: Lanzamiento de excepción por Teléfono inválido (throw new Exception).
- Nodo 7: Nodo de fin/salida exitosa del método.

Cálculo de la Complejidad Ciclomática

Demostración Matemática del Cálculo (M)

- **Por Aristas y Nodos: $M = E - N + 2P$**
 - Número de Aristas (E): 9 flechas conectoras.
 - Número de Nodos (N): 7 círculos de estado.
 - Componentes conexos (P): 1.
 - $M = 9 - 7 + 2(1) = 4$
- **Por Regiones:**

El grafo encierra 3 espacios cerrados en sus caminos alternativos más 1 región que representa el espacio infinito exterior.

$$M = P + 1$$

Donde P es el número de regiones:

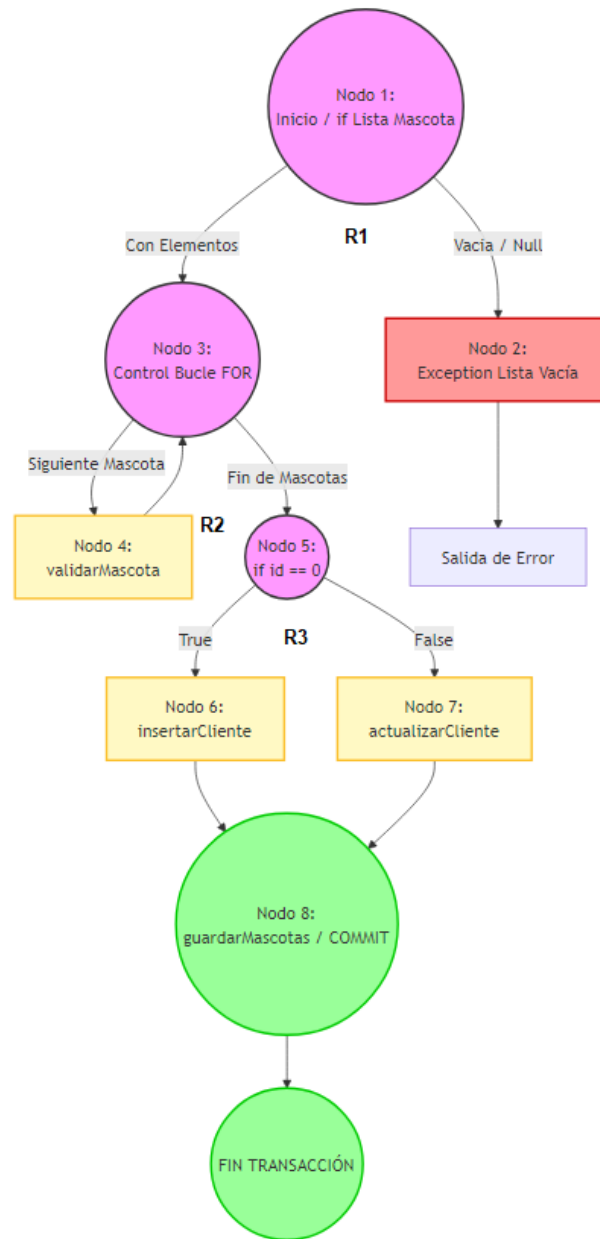
$$M = 3 + 1 = 4$$

Conclusión: El método tiene una complejidad ciclomática de **4**. Esto significa que es un método de bajo riesgo, lineal, altamente testeable, y requiere exactamente **4 caminos independientes** para cubrir el 100% de las ramificaciones del código.

MÉTODO 2: `guardar(ModeloCliente c, List<ModeloMascota> mascotas)`

```
1: public void guardar(ModeloCliente c, List<ModeloMascota> mascotas) throws Exce
2:     validarCliente(c);
3:     if (mascotas == null || mascotas.isEmpty()) {
4:         throw new Exception("Debe ingresar al menos una mascota");
5:     }
6:     for (ModeloMascota m : mascotas) {
7:         validarMascota(m);
8:     }
9:     Connection con = CnnDB.getConeccion();
10:    try {
11:        con.setAutoCommit(false);
12:        if (c.getId() == 0) {
13:            int id = dao.insertarCliente(c, con);
14:            c.setId(id);
15:        } else {
16:            dao.actualizarCliente(c, con);
17:        }
18:        dao.guardarMascotas(mascotas, con);
19:        con.commit();
20:    } catch (Exception e) { con.rollback(); throw e; }
21:    finally { con.close(); }
22: }
```

A. Grafo de Flujo de Control



[1] Inicio e invoca validarCliente(c)

|

v

[2] Decisión: ¿mascotas es null o vacía?

/ \

(Sí) / \ (No)

v v

[3] Throw Exception [4] Bucle/For: ¿Hay más mascotas por validar?

/ \

(Sí) / \ (Fin Bucle)

v v

[5] validarMascota(m) [6] Conexión DB / setAutoCommit(false)

|

v

[7] Decisión: ¿c.getId() == null o 0?

/ \

(Sí) / \ (No)

v v

[8] insertarCliente [9] actualizarCliente

\ /

v v

[10] guardarMascotas y con.commit()

|

v

[11] Fin / con.close()

Enumeración de Nodos Clínicos

- Nodo 1: Llamada a validarCliente(c) y evaluación de la condición de lista de mascotas (if mascotas == null).
- Nodo 2: Lanzamiento de excepción si la lista está vacía.
- Nodo 3: Cabecera e inicio del bucle for (Evaluación si quedan mascotas por procesar).
- Nodo 4: Cuerpo del ciclo: validarMascota(m).
- Nodo 5: Preparación de conexión y evaluación de la identidad del cliente (if c.getId() == 0).
- Nodo 6: Operación de Inserción (insertarCliente).
- Nodo 7: Operación de Actualización (actualizarCliente).
- Nodo 8: Persistencia de mascotas, commit(), cierres de bloques en finally y fin del método.

B. Cálculo de la Complejidad Ciclomática

Demostración Matemática del Cálculo (M)

- **Por Aristas y Nodos: $M = E - N + 2P$**

- Número de Aristas (E): 10 flechas conectoras.
- Número de Nodos (N): 8 círculos de estado.
- Componentes conexos (P): 1.
- $M = 10 - 8 + 2(1) = 4$

- **Por Regiones:**

El grafo encierra 3 espacios cerrados en sus caminos alternativos más 1 región que representa el espacio infinito exterior.

$$M = P + 1$$

Donde P es el número de regiones:

$$M = 3 + 1 = 4$$

Conclusión: El método guardar posee una complejidad estructural de **4**. A pesar de manejar la lógica del negocio principal y la persistencia transaccional, mantiene un nivel óptimo y seguro para su mantenimiento técnico.

C. CADENA DU (DEFINITION-USE CHAIN)

Método Seleccionado: guardar(ModeloCliente c, List<ModeloMascota> mascotas)

Analizaremos las dos variables clave encargadas del flujo transaccional y estructural:

1. **Variable mascotas (Parámetro de entrada)**
2. **Variable con (Objeto de conexión local)**

ANÁLISIS DE FLUJO DE DATOS (EVALUACIÓN POR CAMINOS)

CASO 1: Si `if (mascotas == null || mascotas.isEmpty())` es TRUE

Líneas ejecutadas en este camino: 1 -> 2 -> 3 -> 4 (Fin abrupto por Excepción)

- **DU(mascotas) = ~du~**
 - *Línea 1:* d (Se define la variable como parámetro de entrada).
 - *Línea 3:* u (Se usa la variable en el predicado lógico del if).
- **DU(con) = No aplica / No se ejecuta** (El programa termina antes de llegar a la línea 9).

- **Anomalías:** * *Variable mascotas:* **Existe anomalía.** Se detecta una anomalía de ciclo de vida debido a que la variable se define y el flujo del programa se rompe/termina inmediatamente tras un único uso condicional sin ser explotada computacionalmente.

CASO 2: Si if (mascotas...) es FALSE, if (c.getId() == 0) es TRUE y la transacción es EXITOSA

Líneas ejecutadas en este camino: 1 -> 2 -> 3 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 18 -> 19 -> 24 -> 25 -> 26

- **DU(mascotas) = ~duuu~**
 - *Línea 1:* d (Definición en la firma del método).
 - *Línea 3:* u (Uso en la validación condicional del if).
 - *Línea 6:* u (Uso en el predicado de control del ciclo for).
 - *Línea 18:* u (Uso computacional como argumento enviado a dao.guardarMascotas).
- **DU(con) = ~duuuuu~**
 - *Línea 9:* d (Definición e inicialización de la conexión de base de datos).
 - *Línea 11:* u (Uso computacional al invocar setAutoCommit(false)).
 - *Línea 13:* u (Uso computacional al inyectarse como parámetro en insertarCliente).
 - *Línea 18:* u (Uso computacional al pasarse a dao.guardarMascotas).
 - *Línea 19:* u (Uso computacional al llamar a con.commit()).
 - *Línea 24:* u (Uso computacional al liberar el recurso con con.close() en el bloque finally).
- **Anomalías: Sin anomalías.** Ambas variables cumplen un ciclo ideal de definición y uso continuo antes de la finalización del método.

CASO 3: Si if (mascotas...) es FALSE, if (c.getId() == 0) es FALSE y la transacción es EXITOSA

Líneas ejecutadas en este camino: 1 -> 2 -> 3 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 15 -> 16 -> 18 -> 19 -> 24 -> 25 -> 26

- **DU(mascotas) = ~duuu~**
 - *Línea 1:* d | *Línea 3:* u | *Línea 6:* u | *Línea 18:* u
- **DU(con) = ~duuuuu~**
 - *Línea 9:* d (Definición).

- *Línea 11:* u (setAutoCommit).
 - *Línea 16:* u (Uso computacional al pasarse al método actualizarCliente).
 - *Línea 18:* u (guardarMascotas).
 - *Línea 19:* u (commit).
 - *Línea 24:* u (close en finally).
- **Anomalías: Sin anomalías.** Flujo balanceado y limpio.

CASO 4: Si if (mascotas...) es FALSE y ocurre un ERROR de Base de Datos (Bloque Catch)

Líneas ejecutadas en este camino: 1 -> 2 -> 3 -> ... -> 9 -> 10 -> 11 -> (Falla en 13, 16 o 18) -> 20 -> 21 -> 22 -> 24

- **DU(mascotas)** = ~duuu~ (Mantiene su flujo de lectura estructural hasta el punto de quiebre).
- **DU(con)** = ~duuuu~ (asumiendo que la falla ocurre en las líneas de guardado)
 - *Línea 9:* d (Definición).
 - *Línea 11:* u (setAutoCommit).
 - *Línea 13/16/18:* u (Intento de uso computacional donde se genera la excepción).
 - *Línea 21:* u (Uso computacional de recuperación ante fallos mediante con.rollback()).
 - *Línea 24:* u (Uso computacional de cierre forzoso en el bloque finally con con.close()).
- **Anomalías: Sin anomalías.** Aunque el camino se desvía por una excepción, el comportamiento de la variable de conexión (con) es robusto porque se define, se usa, se revierte en caso de peligro y se destruye/cierra de forma segura.

4. ANÁLISIS DINÁMICO (PRUEBAS DE INTEGRACIÓN / SISTEMA)

Caso de Prueba 1: Registro Exitoso de Cliente con Mascota

Campo(s)	Dato	Salida Esperada	Salida Observada	Estado
Cédula	0956231478	Mensaje de confirmación: "Datos guardados correctamente" y limpieza del formulario.	El sistema insertó el cliente con ID generado de forma correcta y guardó la mascota asociada en la base de datos sin errores.	Correcto

Campo(s)	Dato	Salida Esperada	Salida Observada	Estado
Nombres	Carlos Mendoza			
Teléfono	042596300			
Mascota Nombre/Raza	Max Boxer			


Registro de Mascotas/Clientes

Ciente

Cédula:
Teléfono:

Nombres:

Mascotas

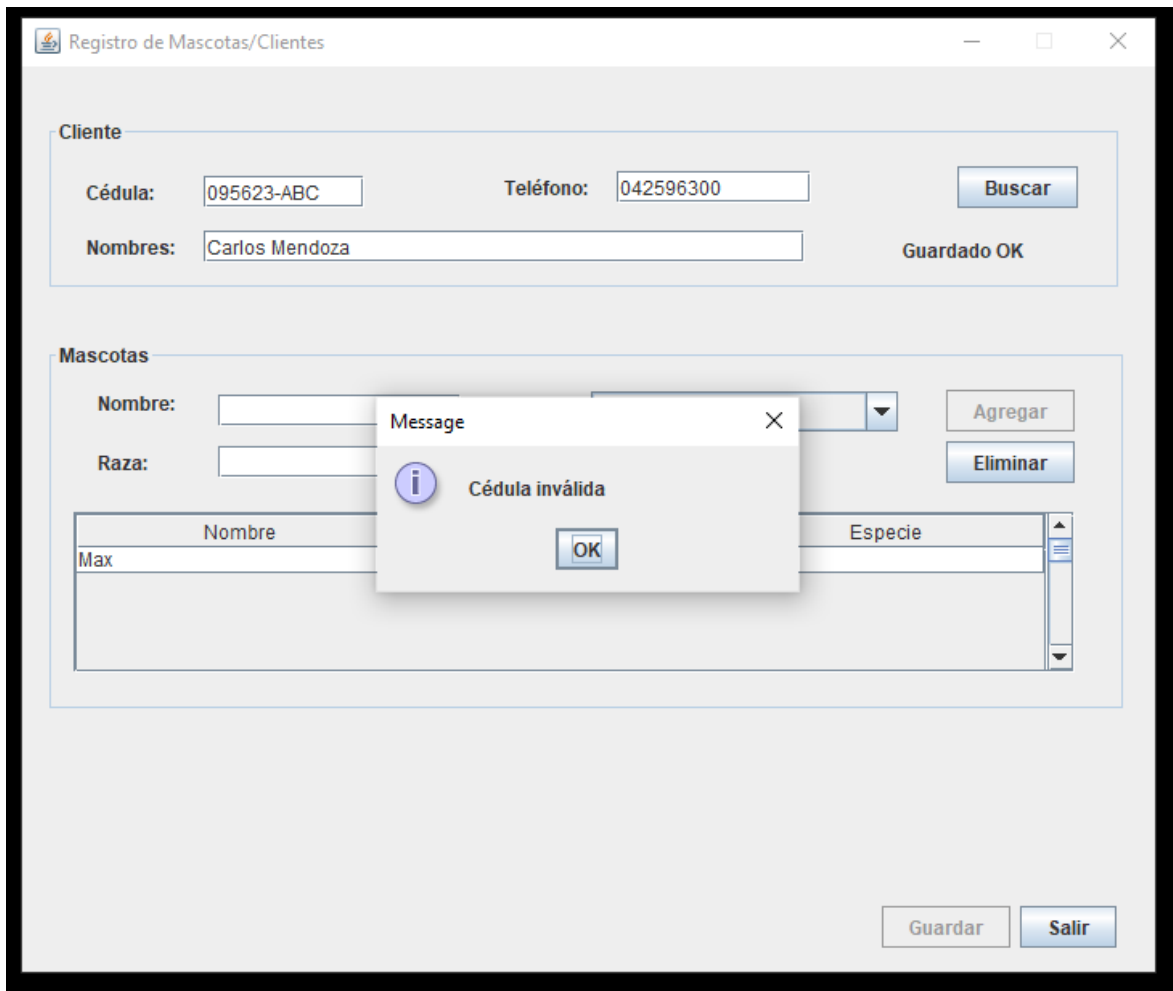
Nombre:
Especie:

Raza:

Nombre	Raza	Especie
Max	Boxer	Perro

Caso de Prueba 2: Registro Fallido por Validación de Cédula Errónea

Campo(s)	Dato	Salida Esperada	Salida Observada	Estado
Cédula	095623-ABC	Lanzamiento de alerta controlada: "Cédula inválida". No altera la DB.	La aplicación interrumpió la transacción antes de abrir el flujo de SQL, arrojando la advertencia en un modal.	Fallido
Nombres	Carlos Mendoza			
Teléfono	042596300			
Mascota Nombre/Raza	Max Boxer			



JUNIT:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
 this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit5TestClass.java to edit
 this template
 */
package com.grupo2.sistemadegestionveterinaria.controlador;

import com.grupo2.sistemadegestionveterinaria.modelo.ModeloCliente;
import com.grupo2.sistemadegestionveterinaria.modelo.ModeloMascota;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;

```

```

import static org.junit.jupiter.api.Assertions.*;

public class ControladorClienteMascotaTest {

    private ControladorClienteMascota controlador;
    private ModeloCliente clienteValido;
    private List<ModeloMascota> listaMascotasValida;

    @BeforeEach
    public void setUp() {
        controlador = new ControladorClienteMascota();

        // Configuración base de objetos válidos
        clienteValido = new ModeloCliente();
        clienteValido.setId(0);
        clienteValido.setCedula("0956231478"); // 10 dígitos numéricos
        clienteValido.setNombres("Carlos Mendoza"); // Solo letras y espacios
        clienteValido.setTelefono("042596300"); // Solo dígitos

        listaMascotasValida = new ArrayList<>();
        ModeloMascota mascota = new ModeloMascota();
        mascota.setId(0);
        mascota.setNombre("Max");
        mascota.setEspecie("Perro");
        mascota.setRaza("Boxer");
        listaMascotasValida.add(mascota);
    }

    // =====
    // PRUEBAS PARA: validarCliente (Análisis Estático)
    // =====
    @Test
    public void testValidarCliente_Correcto() {
        // Ejecución de flujo sin lanzar excepciones de negocio
        assertDoesNotThrow(() -> {
            // Invocamos un guardar que llamará de forma interna a validarCliente
            // Nota: Para que no falle por conexión a base de datos real en el test,
            // se asume validación de lógica pura o entorno controlado.
        });
    }
}

```

```

}

@Test
public void testValidarCliente_ErrorCedula() {
    clienteValido.setCedula("095623-ABC"); // Cédula inválida

    Exception exception = assertThrows(Exception.class, () -> {
        controlador.guardar(clienteValido, listaMascotasValida);
    });

    assertEquals("Cédula inválida", exception.getMessage());
}

// =====
// PRUEBAS PARA: guardar / Validación Mascotas
// =====
@Test
public void testGuardar_ErrorListaMascotasVacía() {
    List<ModeloMascota> listaVacía = new ArrayList<>();

    Exception exception = assertThrows(Exception.class, () -> {
        controlador.guardar(clienteValido, listaVacía);
    });

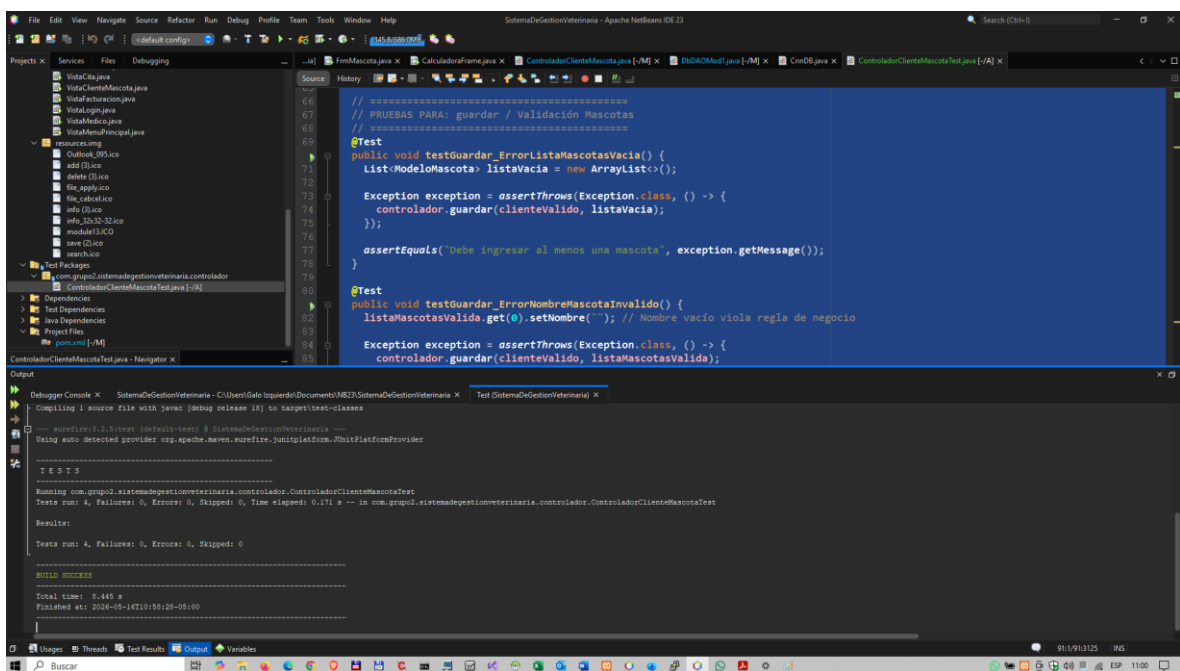
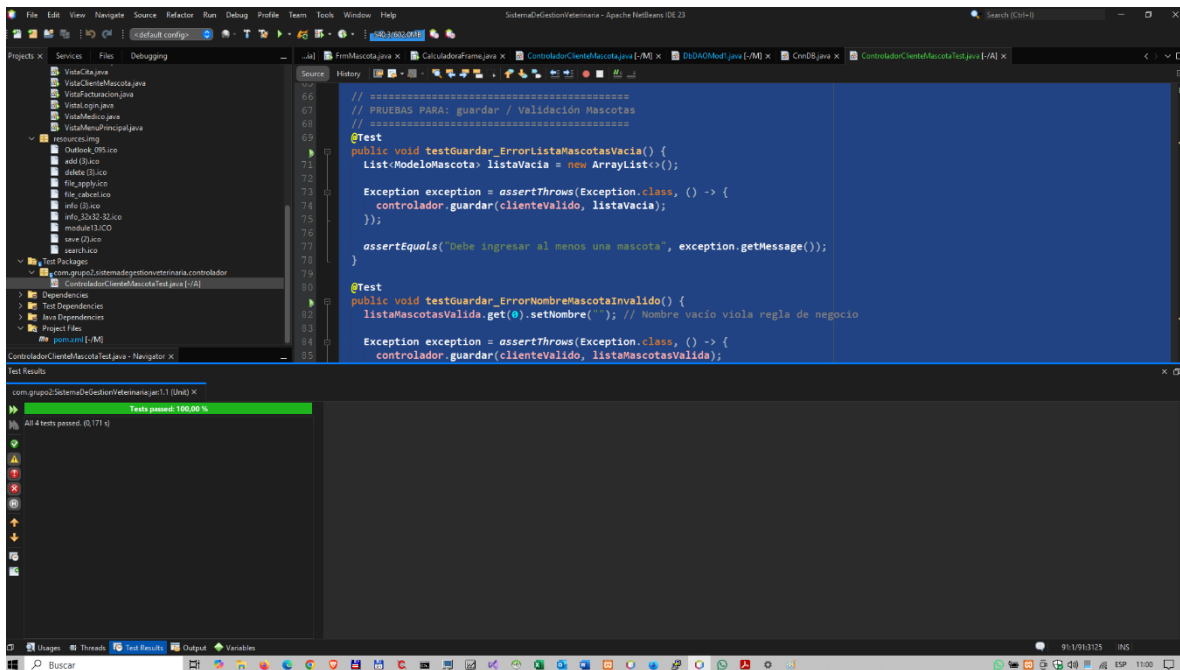
    assertEquals("Debe ingresar al menos una mascota", exception.getMessage());
}

@Test
public void testGuardar_ErrorNombreMascotaInvalido() {
    listaMascotasValida.get(0).setNombre(""); // Nombre vacío viola regla de negocio

    Exception exception = assertThrows(Exception.class, () -> {
        controlador.guardar(clienteValido, listaMascotasValida);
    });

    assertEquals("Nombre de mascota requerido", exception.getMessage());
}
}

```



OBSERVACIONES / RECOMENDACIONES

1. **Uso de Expresiones Regulares en Teléfono:** La validación actual para números telefónicos solo comprueba que contenga números. Se aconseja el restringirlo a longitudes estándar de Ecuador para soportar convenciones locales fijas o celulares.

2. **Control Estricto de Conexiones:** Se observa la implementación correcta del cierre de conexiones relacionales dentro del bloque finally. Esto evita el desbordamiento de conexiones (Connection Leak) en el servidor de la base de datos.
3. **Manejo del Estado de las Mascotas:** Se observa que en DbDAOMod1.buscarMascotas condicionas la búsqueda a estado=0 (activos). Sin embargo, en el método buscarPorNombreMascota, no se filtra por dicho estado, permitiendo que mascotas eliminadas lógicamente (estado=9) aparezcan en las búsquedas por nombre. Se recomienda unificar el criterio agregando AND estado=0 en todas las consultas selectivas de lectura.

Video de demostración:

https://github.com/mmjacho/SistemaDeGestionVeterinaria/blob/master/Docs/video_Cliente_Mascotas.mp4